

Memoria ADSOF - P4

Alejandro Seguido

Rubén García

Apartado 1. Sensores meteorológicos.

Se ha diseñado una jerarquía de clases basada en una clase abstracta *Sensor*, que encapsula el comportamiento y estado común a todos los sensores del sistema. Esta decisión permite aplicar herencia para especializar el comportamiento en los distintos tipos de sensores, manteniendo una única definición de los elementos comunes, lo que mejora la reutilización y la mantenibilidad.

A partir de esta clase base, se han definido tres especializaciones: *SensorTemperatura*, *SensorHumedad* y *SensorPresion*. Cada una de ellas define su rango operativo válido mediante atributos constantes (inmutables), ya que dichos rangos vienen fijados por el enunciado y son comunes a todas las instancias de un mismo tipo. Esta decisión evita inconsistencias y garantiza que todas las instancias de un tipo compartan las mismas restricciones.

Generación de identificadores

El identificador único de cada sensor se genera automáticamente siguiendo el formato especificado en el enunciado (TIPO-NNNN). Para ello, se ha implementado un mapa estático (*Map<String, Integer>*) que mantiene un contador independiente por cada tipo de sensor. De esta forma, se garantiza la unicidad del identificador dentro de cada tipo.

La lógica de generación se encapsula en un método privado de la clase base, evitando duplicación de código en las subclases.

Gestión de unidades

Con el objetivo de favorecer la extensibilidad, se ha definido una interfaz *Unidad* que abstrae el comportamiento común de las unidades de medida. A partir de ella, se han implementado distintas enumeraciones (una por magnitud física).

Este enfoque separa claramente el concepto de tipo de sensor del de unidad de medida, mejorando la cohesión del diseño.

Calibración y validación de lecturas

Cada sensor gestiona su propio estado de calibración mediante:

- Fecha de última calibración.
- Duración de validez de la calibración.
- Atributo boolean *calibrado*.

El método *realizarMedicion()* utiliza excepciones especializadas para garantizar que solo se procesen datos fiables:

- Caducidad: Lanza *SensorDescalibradoPorCaducidadException* si se detecta que el periodo de validez ha expirado antes de iniciar la lectura.
- Fuera de Rango: Lanza *SensorDescalibradoPorRangoException* si el valor resultante no es válido para el sensor. Esta excepción es clave, ya que su ejecución

marca automáticamente al sensor como descalibrado, bloqueando lecturas posteriores.

Estación meteorológica

Se ha implementado una clase *EstacionMeteorologica* como componente central encargado de la gestión de sensores. Internamente, utiliza varias estructuras de datos:

- Un Map<String, Sensor> para acceso eficiente por identificador.
- Un Map<String, List<Sensor>> para agrupación por tipo.
- Un Map<String, LocalDate> para registrar la fecha de instalación.

Esta combinación permite optimizar los distintos casos de uso requeridos (búsqueda por ID, por tipo, listado completo).

Obtención de datos

La estación implementa dos mecanismos de lectura:

- Lectura puntual (*realizarLecturaPuntual*): realiza una medición sobre un conjunto de sensores en una única invocación. Aunque la ejecución es secuencial, se considera “simultánea” a nivel lógico, ya que se trata de una única operación coordinada por la estación.
- Lectura periódica (*comprobarYRealizarLecturaPeriodica*): no se ha implementado como un proceso autónomo, sino como un mecanismo de comprobación bajo demanda. Es decir, cada vez que se invoca el método, se verifica si ha transcurrido el período configurado desde la última medición y, en caso afirmativo, se realiza la lectura.

Esta decisión evita el uso de concurrencia (hilos o temporizadores), reduciendo la complejidad del sistema y siendo suficiente para los requisitos planteados en el enunciado.

Apartado 2. Estrategias de simulación de lecturas de sensores.

Para dotar al sistema de la capacidad de simular diversos comportamientos ambientales y condiciones de fallo, se ha implementado el patrón de diseño Strategy. Esta decisión permite desacoplar la lógica de generación de datos de la estructura física del sensor, facilitando la extensibilidad y permitiendo cambiar el comportamiento de simulación en tiempo de ejecución sin modificar las clases de la jerarquía Sensor.

Interfaz de estrategia

Se ha diseñado la interfaz *EstrategiaLectura* como la base común para todos los algoritmos de simulación. Esta interfaz define un único método, *generarValor(Sensor s)*, que recibe como parámetro la instancia del sensor que solicita la lectura. El paso del objeto sensor por parámetro es una decisión de diseño clave, ya que permite que las estrategias consulten dinámicamente los límites operativos (*minRango*, *maxRango*) o el histórico de datos del sensor específico para realizar sus cálculos.

Estrategias implementadas

Siguiendo los requisitos, se han desarrollado tres implementaciones concretas de la interfaz:

- **Estrategia Aleatoria (*EstrategiaAleatoria*):** Permite generar valores dentro del rango de operación normal del sensor. Adicionalmente, incorpora un atributo *probFueraRango* que simula fallos de hardware o condiciones extremas, generando valores por encima del máximo permitido con una probabilidad configurable.
- **Estrategia de Valor Cercano (*EstrategiaValorCercano*):** Diseñada para simular la inercia física de las magnitudes medidas (donde los valores no cambian bruscamente entre lecturas). Utiliza el último valor registrado en el histórico del sensor y aplica una variación porcentual definida por el usuario.
- **Estrategia Histórica (*EstrategiaHistorica*):** Simula un comportamiento de tendencia o estabilidad climática. Calcula la media de todas las mediciones previas almacenadas en el sensor y genera un nuevo valor aleatorio en un entorno cercano a dicha media.

En las estrategias basadas en el histórico (*ValorCercano* e *Historica*), se ha implementado una lógica de seguridad: si el sensor no dispone de lecturas previas (primera medición), la estrategia devuelve automáticamente el valor medio del rango operativo ($\text{minRango} + \text{maxRango} / 2$) para asegurar la continuidad del sistema.

Integración y valores por defecto

La relación entre el sensor y su estrategia se gestiona mediante composición. Para cumplir con el requisito de "estrategia por defecto", los constructores de las clases de sensores se han sobrecargado. Si el usuario no proporciona una estrategia específica al instanciar un sensor, el sistema asigna automáticamente una *EstrategiaAleatoria* con una probabilidad de error configurada al 5%.

Si en el futuro se requiriesen nuevos modelos de simulación, bastaría con crear una nueva clase que implemente *EstrategiaLectura* sin alterar el código existente de la estación o los sensores.

Apartado 3.

Para este apartado hemos creado la clase `ProcesadorDatos` la cual guarda en un treemap los datos registrados asociados a un sensor con su fecha. El motivo por el cual usar el treemap es para ordenar los datos según su fecha.

Un sensor puede tener asociado un procesador de datos para guardar los datos que vaya leyendo. Hemos decidido que tanto el sensor como el procesador de datos van a guardar la información, esto es por si en un futuro se cambia de procesador de datos se pueda tener ese histórico de dicho sensor.

El procesador a su vez guarda un conversor, que por defecto es el conversor identidad. El conversor se utiliza para guardar los datos en el mapa del procesador en la unidad de destino del conversor.

Hemos creado la interfaz `conversor` con unos métodos básicos, `unidad origen`, `unidad destino`, `convertir` y `concatenar`. Los dos primeros son para guardar las unidades, el siguiente sirve para utilizar la fórmula de conversión de unidades y el último para poder concatenar varios conversores.

Hemos creado varias clases que implementan esta interfaz, `conversor identidad`, `kelvin a fahrenheit`, `fahrenheit a celsius` y los respectivos inversos, pudiendo pasar cualquier conversión gracias al conversor compuesto.

El conversor compuesto guarda 2 conversores, y a la hora de convertir se hace una llamada a sí mismo con el valor intermedio.

Apartado 4.

Con el fin de dotar al sistema de una mayor robustez y coherencia, se ha diseñado una jerarquía de excepciones personalizadas. Estas heredan de `Exception` o `RuntimeException` según la naturaleza del error, pudiendo concretar que tipo de errores han saltado y poderlos gestionar de forma coherente. Entre las distintas excepciones tenemos:

Validación de rangos y duplicidades

- **ValoresRangoException:** Se lanza durante la instancia del sensor para asegurar la integridad de los límites operativos. Esta decisión de diseño impide la creación de objetos donde el valor de `minRango` sea superior o igual a `maxRango`.
- **IdDuplicadoException:** Garantiza la unicidad de los registros en la `EstacionMeteorologica`. Al intentar añadir un nuevo sensor, el sistema verifica que su identificador no exista previamente en el mapa de sensores, evitando la sobrescritura accidental de dispositivos.

Control de calibración y flujo de datos

- **SensorDescalibradoException:** Se dispara si la desviación (offset) detectada supera un umbral crítico. Este límite se calcula dinámicamente como un porcentaje del rango total del sensor, permitiendo su aplicación proporcional a cualquier tipo de magnitud física.
- **CambioBruscoException:** Diseñada para detectar anomalías o lecturas erráticas. Si la diferencia entre la nueva medición y el último registro supera el `umbralCambio` configurado, el sistema aborta la operación para alertar sobre una posible avería técnica.
- **ConversionErroneaException:** Garantiza la coherencia dimensional dentro del `ProcesadorDatos`. Esta excepción previene una transformación matemática inválida

si se intenta inyectar un Conversor cuya unidad de origen no coincida con la unidadBase del procesador.

Además, la EstacionMeteorologica integra un registro de registroAlertas. Cuando ocurren excepciones críticas durante el funcionamiento, estas se capturan y se introducen en una lista junto a su marca de tiempo (LocalDateTime), facilitando la generación de un histórico de incidencias para su posterior revisión.

Apartado 5.

Se ha diseñado una arquitectura basada en interfaces con el objetivo de dotar al sistema de la capacidad de formatear la información generada por la estación, siendo este diseño extendible. Esta decisión de diseño evita que la clase EstacionMeteorologica tenga responsabilidades de visualización o dependa de formatos específicos como HTML o Markdown, mejorando la mantenibilidad y cohesión del componente.

Interfaz IDocumento: Para abstraer el contenido estructural que cualquier elemento exportable debe proporcionar, se ha definido la interfaz IDocumento. La clase EstacionMeteorologica implementa esta interfaz, lo que permite la obtención de los datos como el título (getTituloDocumento), secciones (getTituloSeccion) y colecciones de datos (getParrafos y getListas).

Interfaz Formateador : Se ha definido la interfaz IFormateador para establecer como se formatea un objeto de tipo IDocumento. Las implementaciones concretas, FormateadorHTML y FormateadorMarkdown, son las responsables exclusivas de gestionar las etiquetas de marcado (<h1>, , o #, -, etc.) al iterar sobre las estructuras de datos provistas. Este enfoque permite un diseño escalable y que se puedan crear otros formateadores sin tener que cambiar el código.

Pruebas:

Prueba del formateador, sacando el output en ambos formatos solicitados:

```
ruben@ruben-Legion-5-Pro:~/ADSOF/Practica4$ javac -d bin $(find src -name "*.java")
ruben@ruben-Legion-5-Pro:~/ADSOF/Practica4$ java -cp bin main.Main
Realizando mediciones...

SALIDA EN FORMATO HTML
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Estación Meteorológica: Madrid Centro</title>
</head>
<body>
  <h1>Madrid Centro</h1>
  <p>Ubicación: -3.7038, 40.4168</p>
  <p>Sensores instalados: 2</p>
  <p>Última lectura: Ninguna</p>
  <p>Sensores activos</p>
  <ul>
    <li>HUM-0001 (%): [45.00] -- MIN: 45.00 MAX: 45.00 AVG: 45.00</li>
    <li>TEMP-0001 (°C): [22.50] -- MIN: 22.50 MAX: 22.50 AVG: 22.50</li>
  </ul>
  <p>Alertas activas</p>
  <ul>
    <li>[2026-04-13T12:23:47.758595575] Alerta en HUM-0001: Alerta en sensor HUM-0001: Offset del sensor descalibrado</li>
    <li>[2026-04-13T12:23:47.758696464] Alerta en TEMP-0001: Alerta en sensor TEMP-0001: Offset del sensor descalibrado</li>
  </ul>
</body>
</html>

SALIDA EN FORMATO MARKDOWN
# Estación Meteorológica: Madrid Centro

## Madrid Centro

Ubicación: -3.7038, 40.4168

Sensores instalados: 2

Última lectura: Ninguna

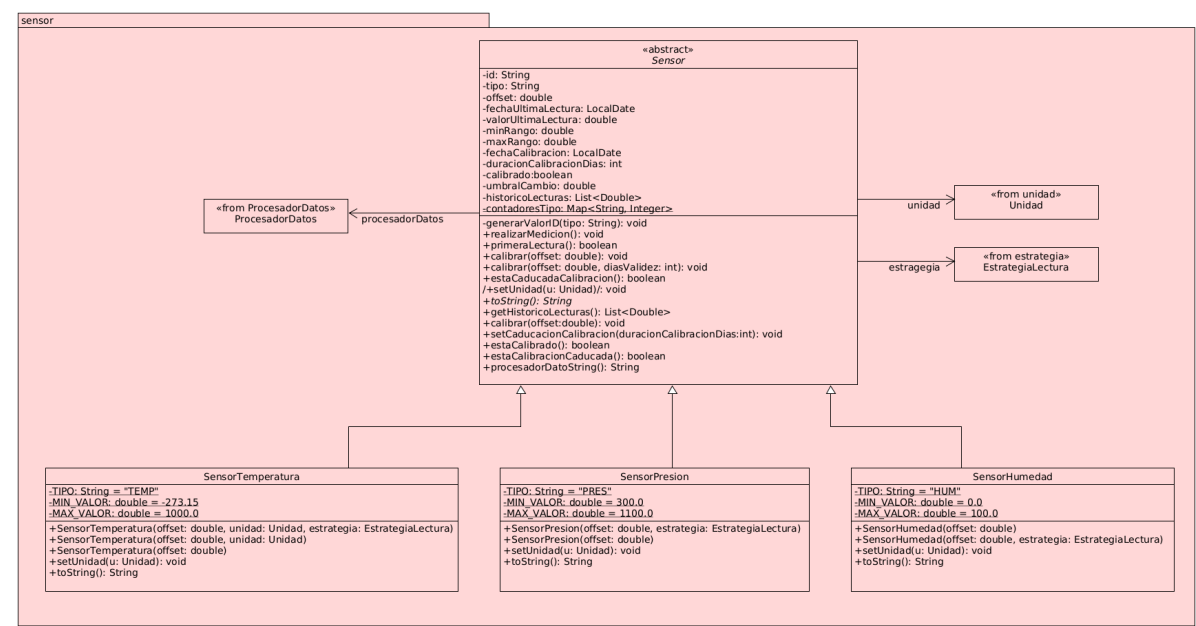
### Sensores activos
- HUM-0001 (%): [45.00] -- MIN: 45.00 MAX: 45.00 AVG: 45.00
- TEMP-0001 (°C): [22.50] -- MIN: 22.50 MAX: 22.50 AVG: 22.50

### Alertas activas
- [2026-04-13T12:23:47.758595575] Alerta en HUM-0001: Alerta en sensor HUM-0001: Offset del sensor descalibrado
- [2026-04-13T12:23:47.758696464] Alerta en TEMP-0001: Alerta en sensor TEMP-0001: Offset del sensor descalibrado
ruben@ruben-Legion-5-Pro:~/ADSOF/Practica4$
```

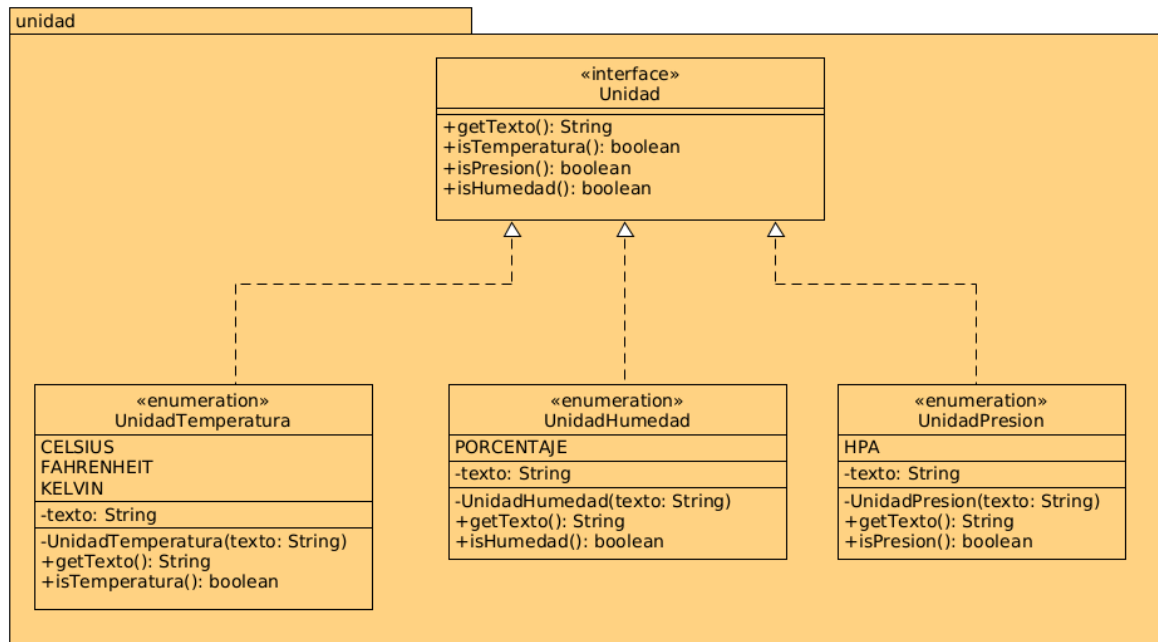
Para probar cada clase por separado se han implementado unos junit, que están en la carpeta correspondiente a test.

Diagrama de diseño.

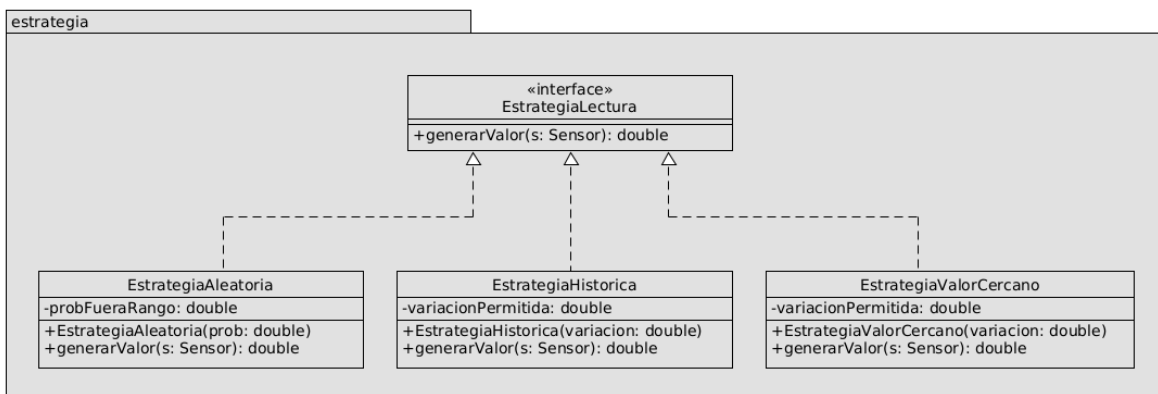
Paquete sensor:



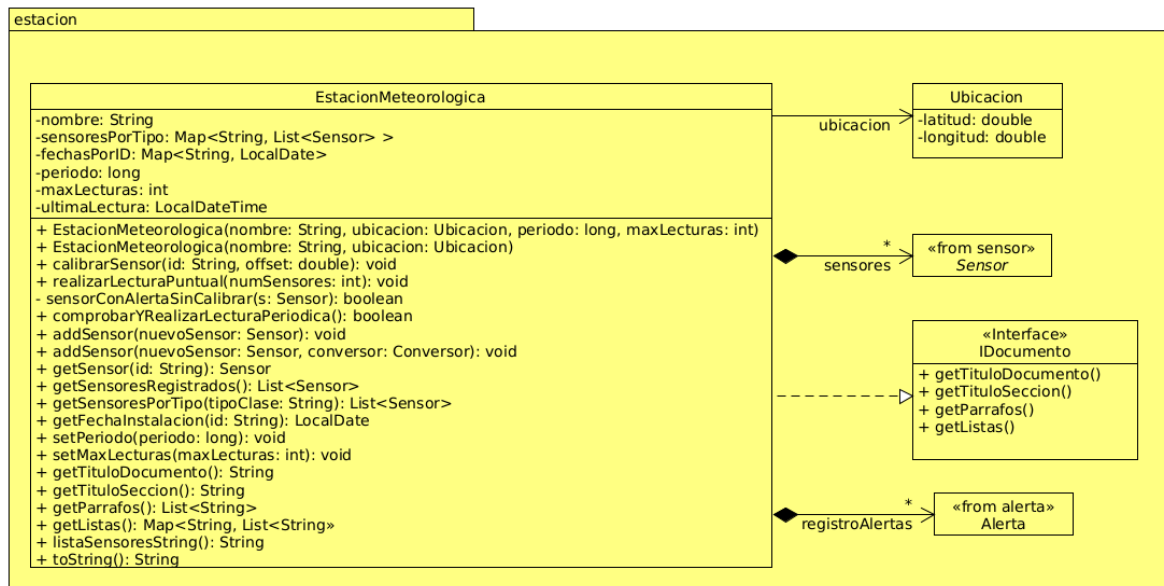
Paquete unidad:



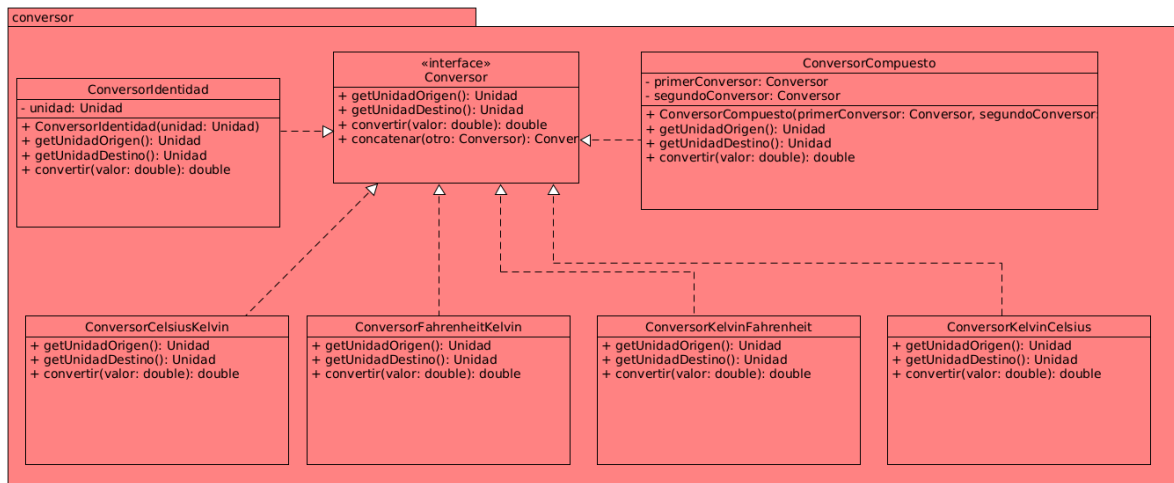
Paquete estrategia:



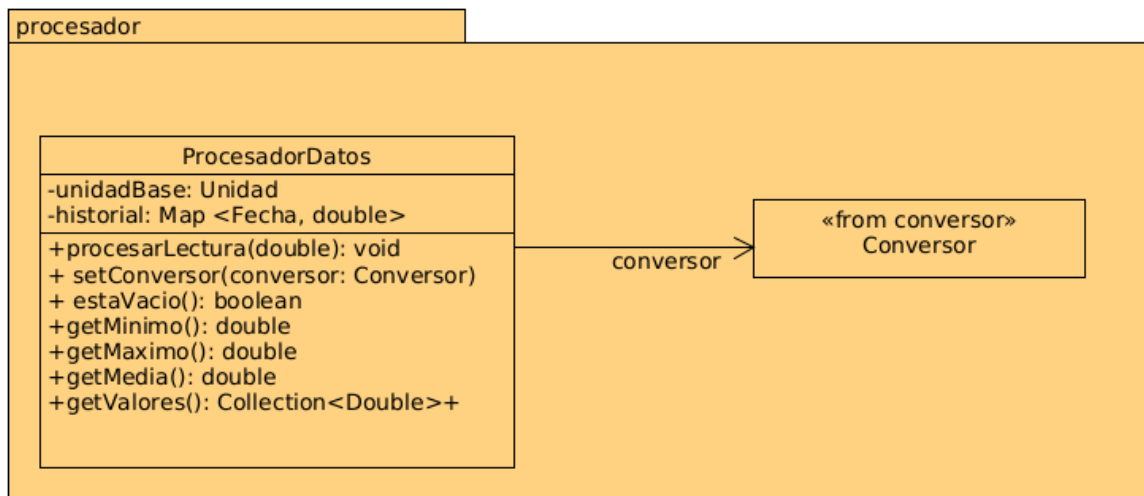
Paquete estacion:



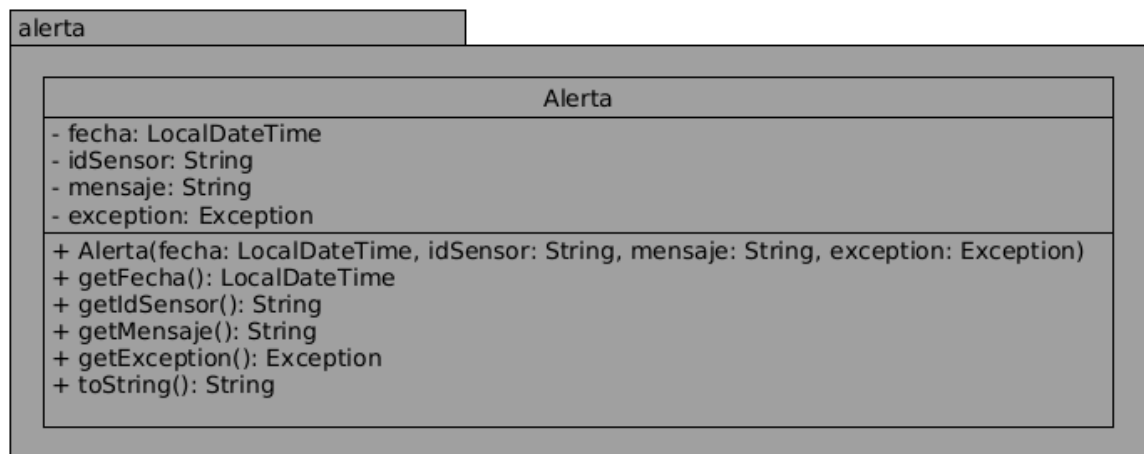
Paquete conversor:



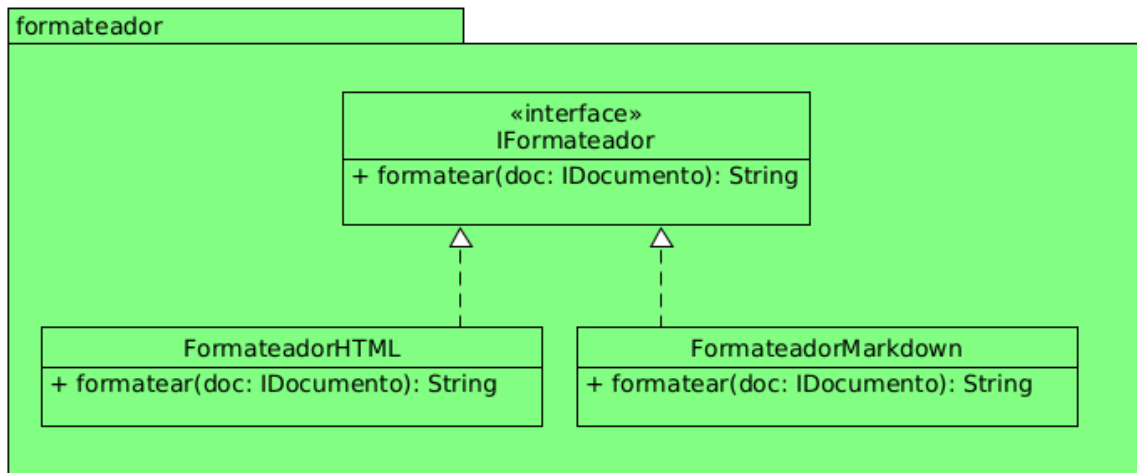
Paquete procesador:



Paquete alerta:



Paquete formateador:



Paquete excepcion:

